



Pilha



Prof. Rafael Mesquita
rgm@cin.ufpe.br

O que veremos nesta aula?



- Definição de Pilha
- Implementação utilizando vetores
- Implementação utilizando Lista Encadeada
- Exemplos de uso

Definição de Pilha

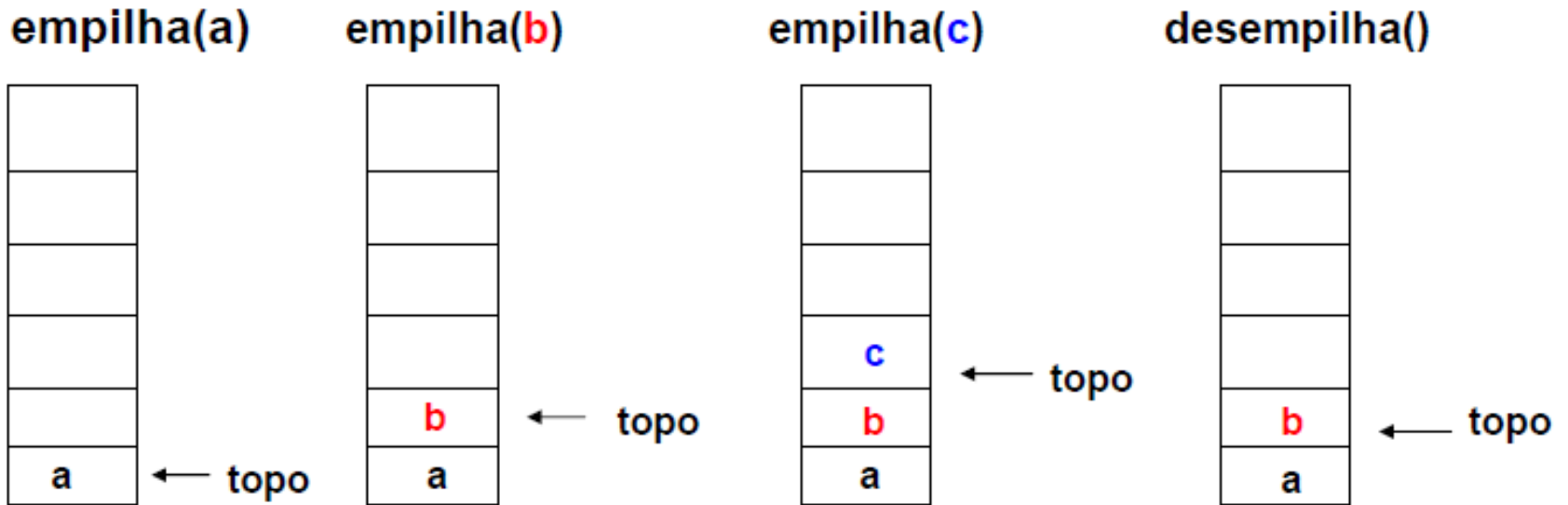
- Pilha
- Estrutura de dados dinâmica
 - ▣ Inserção de elementos acontece sempre no topo da pilha
 - ▣ Remoção de elementos também acontece sempre no topo da pilha
 - ▣ L.I.F.O.
 - *Last in, first out!*
 - ▣ Pode-se inserir qualquer elemento. No entanto, o elemento removido deve ser sempre o último elemento inserido

Definição de Pilha

- Pilha
- Operações básicas
 - Empilhar
 - `push(novo_elemento)`
 - Insere um novo elemento da pilha
 - Inserção ocorre no topo da pilha
 - Desempilhar
 - `pop()`
 - Retira um elemento da pilha
 - Elemento do topo da pilha é removido

Definição de Pilha

□ Funcionamento



Exemplos de uso

- ❑ Operações de “desfazer” (editores de texto)
- ❑ Pilha de execução de funções (compiladores)
- ❑ Avaliação de expressões aritméticas (compiladores)

Exemplos de uso

□ Pilha de execução de funções

```
void main()
{
    int n = 5;
    int r;
    r = fat(n);
    printf("fatorial de %d = %d", n, r);
    getchar();
}
```

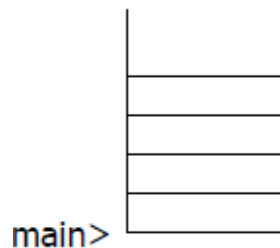
```
#include <stdlib.h>

int fat(int n)
{
    int f = 1;
    while (n != 0)
    {
        f *= n;
        n--;
    }
    return f;
}
```

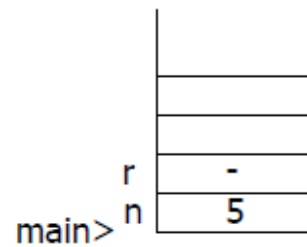
Exemplos de uso

□ Pilha de execução de funções

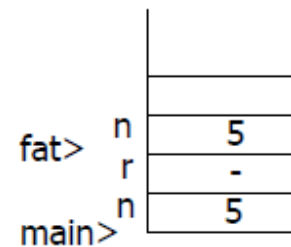
1 – Início do programa:
pilha vazia



2 – Declaração de
variáveis: n,r

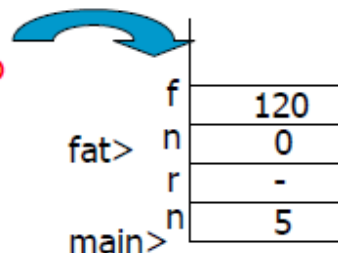


3 – Chamada da função:
empilha variáveis

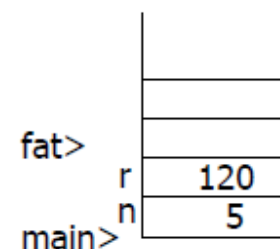


4 – Final do laço

Acesso às variáveis que
estão na função do topo



5 – Retorno da função:
desempilha



Implementação

- Utilizando vetores
- Utilizando lista encadeada

Implementação (vetor)

```
#include <stdlib.h>
#define N_MAX 50

struct pilha {
    int n_elems;
    int vet[N_MAX];
};

typedef struct pilha Pilha;

Pilha* criaPilha()
{
    Pilha* p = (Pilha*)malloc(sizeof(Pilha));
    p->n_elems = 0;
    return p;
}
```

Implementação (vetor)

```
void push(Pilha* pilha, int info)
{
    if (pilha->n_elems >= N_MAX)
    {
        printf("capacidade máxima já atingida! elemento não inserido!\n");
        return;
    }
    pilha->vet[pilha->n_elems] = info;
    pilha->n_elems++;
}

int pilha_vazia(Pilha* pilha)
{
    if (pilha->n_elems == 0)
        return 1;
    else
        return 0;
}
```

Implementação (vetor)

```
int pop(Pilha* pilha)
{
    int elem;
    if (pilha_vazia(pilha))
    {
        printf("\npilha vazia. nenhum elemento removido!\n");
        return 0;
    }

    elem = pilha->vet[pilha->n_elems-1];
    pilha->n_elems--;
    return elem;
}
```

Implementação (vetor)

```
void libera(Pilha* pilha)
{
    free(pilha);
}

void imprime(Pilha* pilha)
{
    printf("\n");
    if (pilha_vazia(pilha))
    {
        printf("pilha vazia!\n");
        return;
    }
    printf("[");
    for(int i = 0; i < pilha->n_elems; i++)
        printf("%d ", pilha->vet[i]);
    printf("]");
}
```

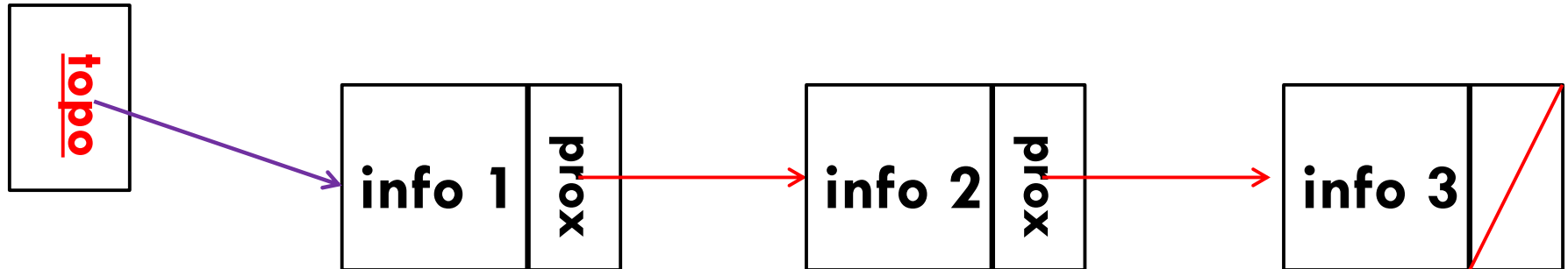
Implementação

- Implementação com vetor não otimiza uso da memória
- Abordagem mais utilizada
 - ▣ Implementação de Pilha utilizando uma Lista Encadeada
 - Regra L.I.F.O. é aplicada sobre uma lista encadeada
 - ▣ Espaço em memória é otimizado
 - Evita-se o desperdício
 - Não é necessária estimativa da quantidade de elementos

Implementação (lista encadeada)

□ Definição da estrutura

pilha

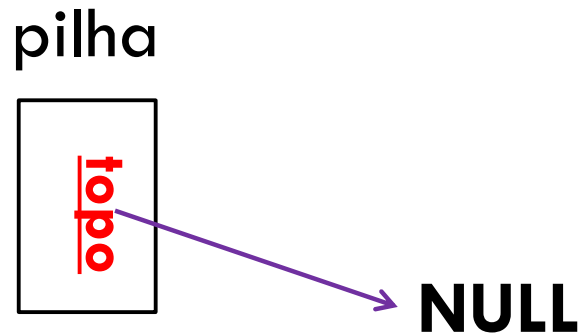


```
typedef struct lista {  
    float info;  
    struct lista* prox;  
} Lista;
```

```
typedef struct pilha {  
    Lista *topo;  
} Pilha;
```

Implementação (lista encadeada)

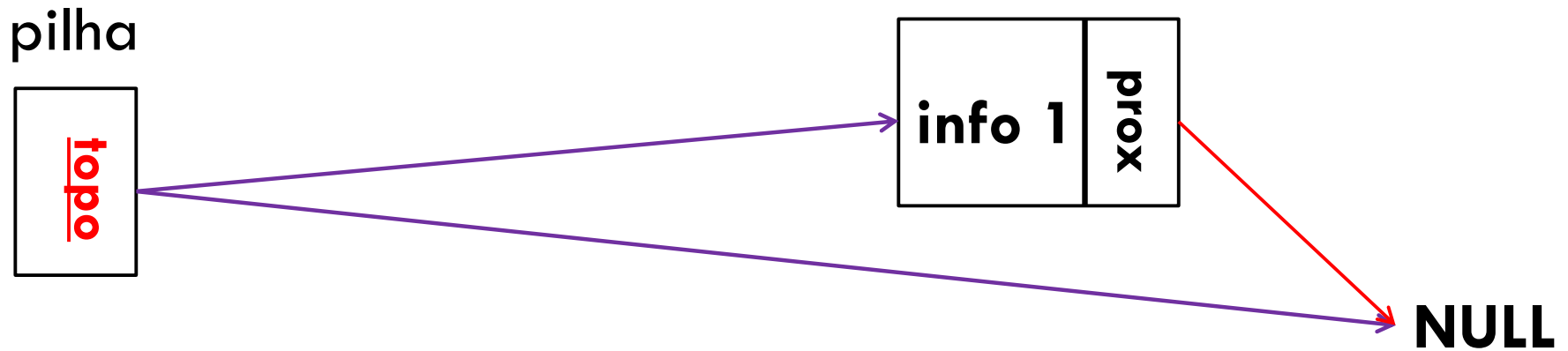
□ Criação da pilha



```
Pilha* cria()
{
    Pilha* p = (Pilha*)malloc(sizeof(Pilha));
    p->topo = NULL;
    return p;
}
```


Implementação (lista encadeada)

□ Inserção na pilha (push)



```
void push(Pilha* p, float valor)
{
    Lista* no = (Lista*)malloc(sizeof(Lista));
    no->info = valor;
    no->prox = p->topo;
    p->topo = no;
}
```

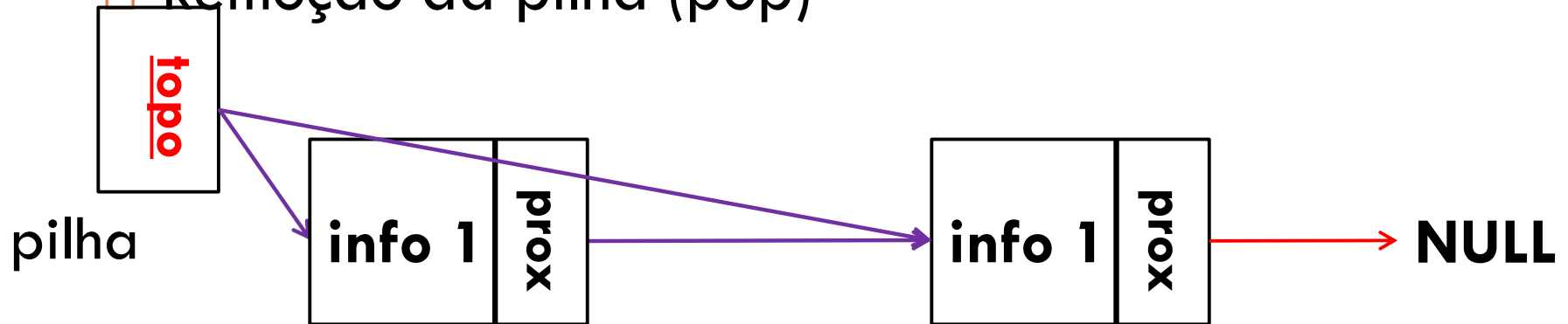
Implementação (lista encadeada)

□ Testa se está vazia

```
int pilha_vazia(Pilha* p)
{
    return p->topo == NULL;
}
```

Implementação (lista encadeada)

Remoção da pilha (pop)



```
void pop(Pilha* p)
{
    if (pilha_vazia(p))
    {
        printf("\npilha vazia!");
        return;
    }
    Lista* aux = p->topo;
    p->topo = p->topo->prox;
    free(aux);
}
```

Implementação (lista encadeada)

□ Liberar memória

```
void libera(Pilha** p)
{
    Lista* l = (*p)->topo;
    while (l != NULL)
    {
        Lista* aux = l->prox;
        free(l);
        l = aux;
    }
    free(*p);
    *p = NULL;
}
```

Implementação (lista encadeada)

□ Imprimir

```
void imprime(Pilha* p)
{
    printf("\n");
    if (p == NULL || p->topo == NULL)
    {
        printf("pilha vazia!");
        return;
    }
    for (Lista* l = p->topo; l != NULL; l = l->prox)
        printf("%f ", l->info);
}
```

Exercícios

1. Crie sua própria implementação de Pilha utilizando uma lista encadeada com as funcionalidades apresentadas nesta aula
2. Implemente uma pilha em C utilizando listas encadeadas. Utilize sua implementação para criar uma função que avalie se uma expressão aritmética (string recebida como parâmetro) é válida em relação ao uso de parenteses.

